Software Requirements Specification for Lake: subtitle describing software

Team #10, CinqGarçons
Ethan Walsh
Colin Chambachan
Matthew Nesbitt
Jinwoo Hong
Gary Qin

October 19, 2026

Table 1: Revision History

Date	**Developer(s)**	**Change**
2026-10-09	Colin Chambachan	Switch from the Volere template to the Meyer template
2026-10-15	Gary Qin	S.9 Formalizations
Date2	Name(s)	Description of changes
...	...	...

Contents

Changes from Original SRS Meyer Template **1**

Glossary **3**

(G) Goals **4**
G.1 Context and Overall Objective . . . 4
G.2 Current Situation . . . 4
G.3 Expected Benefits . . . 4
G.4 Functionality Overview . . . 4
G.5 High-Level Usage Scenarios . . . 4
G.6 Limitations and Exclusions . . . 5
G.7 Stakeholders and Requirements Sources . . . 5
G.8 User Profiles . . . 5

(E) Environment **6**
E.1 Glossary . . . 6
E.2 Environment Components . . . 6
E.3 Constraints . . . 6
E.4 Assumptions . . . 6
E.5 Effects . . . 6
E.6 Invariants . . . 7
E.7 Standards and Legal/Regulatory Factors . . . 7
Environmental Requirements . . . 7

(S) System **8**
S.1 Components . . . 8
S.2 Functionality . . . 9
S.2.1 Coordinator . . . 9
S.2.2 Model Partitioner . . . 12
S.2.3 Inference Engine . . . 12
S.2.4 Tensor I/O . . . 13
S.2.5 Web UI . . . 13
S.2.6 Non-Functional Requirements . . . 14
S.2.6.1 Usability . . . 14
S.2.6.2 Performance . . . 15
S.2.6.3 Look and Feel . . . 15

S.2.6.4 Fault-Tolerance . . . 15
S.2.6.5 Privacy . . . 15
S.2.6.6 Compatibility . . . 16
S.3 Interfaces . . . 16
S.4 Normal Operation . . . 17
S.5 Undesired Event Handling . . . 18
S.6 Detailed Usage Scenarios . . . 18
S.7 Prioritization . . . 19
S.8 Verification and Acceptance Criteria . . . 20
S.9 Formalizations . . . 20
S.9.1 Notation . . . 20
S.9.2 Valid Partitioning Schemes . . . 21
S.9.3 Allocation State Machine . . . 22
S.9.3.1 States. . . . 22
S.9.3.2 Events. . . . 23
S.9.3.3 Guards. . . . 23
S.9.3.4 Transitions. . . . 23
S.9.3.5 Invariant. . . . 24
S.10 Likely Changes to Requirements . . . 24
S.11 Unlikely Changes to Requirements . . . 25

(P) Project **26**
P.1 Roles and Personnel . . . 26
P.2 Imposed Technical Choices . . . 26
P.3 Schedule and Milestones . . . 26
P.4 Tasks and Deliverables . . . 26
P.5 Required Technology Elements . . . 27
P.6 Risk and Mitigation Analysis . . . 27
P.7 Requirements Process and Report . . . 27

Safety and Security Requirements **28**

Requirement Dependencies **29**

1 Appendix — Generative AI Use Disclosure **31**
1.1 AI Tools Used . . . 31
1.2 How and Where Used . . . 31
1.3 Verification of AI-Generated Content . . . 31

2 Appendix — Reflection **32**

List of Tables

1 Revision History i
2 Changes from Template 2
3 Environmental Requirements 7
4 Prioritization of Component Name requirements 20
5 Symbols used in the formalization 21
6 Transition function δ of the allocation state machine 24
7 Project Milestones and Due Dates 26
8 Tasks and Deliverables 27

List of Figures

1 Component diagram. 8

Changes from Original SRS Meyer Template

[Meyer's template is a book chapter, not a course template, so every team adapts it. Record each structural change you make and why, so a reader who knows the original can follow this document. The rows below describe the adaptations already present in this file; add to them as the document evolves. —TPLT]

Table 2: Changes from Template

Change	**Reason**
Glossary (E.1) moved to the front of the document	Terms and acronyms must be defined before first use, and information should live in one place. E.1 now points here.
Requirement tables added to each PEGS chapter	Meyer's template describes requirements in prose. Labelled tables of functional (FR) and non-functional (NFR) requirements give each requirement an ID that can be prioritized, traced and verified.
Added G.8 User Profiles	Meyer lists stakeholders (G.7) but has no place for the distinct characteristics of each user group.
Added E.7 Standards and Legal/Regulatory Factors	Meyer's Environment chapter has no explicit home for standards, licensing or legal constraints.
Added S.4 Normal Operation and S.5 Undesired Event Handling	Separates expected behaviour from error handling instead of mixing both into S.2 Functionality.
Added S.9 Formalizations	Provides a place for mathematical or formal specification of the parts of the system that benefit from it.
Added Safety and Security Requirements	Lets the requirements from the Hazard Analysis be ported into this document.
Added Requirement Dependencies	Shows traceability between requirements visually.
Added References	Meyer's template has no references section.
Added Appendices (Generative AI Use Disclosure, Reflection)	Required by the course.
Added S.10 Likely changes to requirements	Required by the SRS rubric
Added S.11 Unlikely changes to requirements	Required by the SRS rubric

Glossary

[Define every domain term and acronym used in this document, in alphabetical order. Define a term once, here, and use it consistently everywhere else. Where a term has a good external definition, link to it and cite the source. —TPLT]

[Tip from the source template: define a `\hypertarget` for each glossary entry and a macro such as `\newcommand{\Term}{\hyperlink{glossary-term}{Term}}` so each use of the term in the body links back to its definition. —TPLT]

- **Term** – Definition.
- **Acronym (Expansion)** – Definition.

(G) Goals

[The Goals chapter is written for decision makers. It explains why the project exists, in the language of the business or domain, not of the software. It should be readable on its own. —TPLT]

G.1 Context and Overall Objective

[One or two paragraphs: the domain the system lives in, the problem it addresses, and the overall objective in a sentence a sponsor would recognize. —TPLT]

G.2 Current Situation

[How things are done today without the system, and what is wrong or missing. Reference existing tools or processes the system will replace or complement. —TPLT]

G.3 Expected Benefits

[The improvements the system will bring over the current situation, from the point of view of the stakeholders. Where possible, make each benefit measurable so it can be checked later. —TPLT]

- **Benefit** – Description.

G.4 Functionality Overview

[A high-level list of what the system does. Each item should correspond to one or more components in S.1 and requirements in S.2, but stay at the level a non-technical reader can follow. —TPLT]

- **Capability** – Description.

G.5 High-Level Usage Scenarios

[The main ways users will interact with the system, as short narratives (two or three sentences each). Detailed use cases belong in S.6. An activity diagram can be included here if it helps. —TPLT]

- **Scenario 1: Name** – Description.

G.6 Limitations and Exclusions

[What the system will not do. State exclusions explicitly so nobody later assumes they were simply forgotten. —TPLT]

- **Exclusion** – Description and rationale.

G.7 Stakeholders and Requirements Sources

[Stakeholders: every person or group affected by the system, including clients, users, supervisors, and the wider community. Requirements sources: the documents, people, standards, and existing systems the requirements were elicited from. —TPLT]

Stakeholders:

- **Stakeholder** – Their interest in the system.

Requirements Sources:

- **Source** – What it contributed.

G.8 User Profiles

[For each distinct group of hands-on users, describe their domain knowledge, goals, technical literacy, and which parts of the system matter most to them. These profiles justify usability and accessibility requirements later on. —TPLT]

1. Profile Name

- **Domain Knowledge:**
- **Goals:**
- **Technical Literacy:**
- **Rationale:** Which requirements exist for this user.

(E) Environment

[The Environment chapter describes the world the system operates in and cannot change: physical constraints, other systems, rules of the domain. These hold whether or not the system is built. —TPLT]

E.1 Glossary

The glossary has been moved to the front of the document (see the Glossary and the Changes from Original SRS Meyer Template).

E.2 Environment Components

[Elements of the environment that may affect or be affected by the system and the project, including other systems the system must interface with, hardware it runs on, and the people who operate it. —TPLT]

- **Component** – Description and its relationship to the system.

E.3 Constraints

[Obligations and limits imposed on the system by the environment: regulations, physical limits, resource limits, deadlines. These are not design choices; the system must live with them. —TPLT]

- **Constraint** – Description.

E.4 Assumptions

[Properties of the environment that the system relies on but cannot enforce. Each assumption is a risk if it turns out false; cross-reference the Hazard Analysis where relevant. —TPLT]

- **Assumption** – Description.

E.5 Effects

[The ways the system changes its environment once deployed: effects on users, on other systems, on resources. —TPLT]

- **Effect** – Description.

E.6 Invariants

[Properties of the environment that must remain true throughout the system's operation, and that the system must preserve. —TPLT]

- Invariant.

E.7 Standards and Legal/Regulatory Factors

[Coding, documentation, interface or safety standards the project will follow, and legal or regulatory factors such as licensing, intellectual property, and data privacy. Cite each standard. —TPLT]

Standards:

- **Standard** – What it governs and how it applies.

Legal and Regulatory Factors:

- **Factor** – Description.

Environmental Requirements

[Requirements that follow directly from this chapter: installability on the target platforms, resource limits, invariant enforcement, standards compliance, legal compliance. Label each one so it can be referenced from S.7 and S.8. —TPLT]

Table 3: Environmental Requirements

Label	Requirement
NFR.E.1	The system shall ...
NFR.E.2	The system shall ...

(S) System

S.1 Components

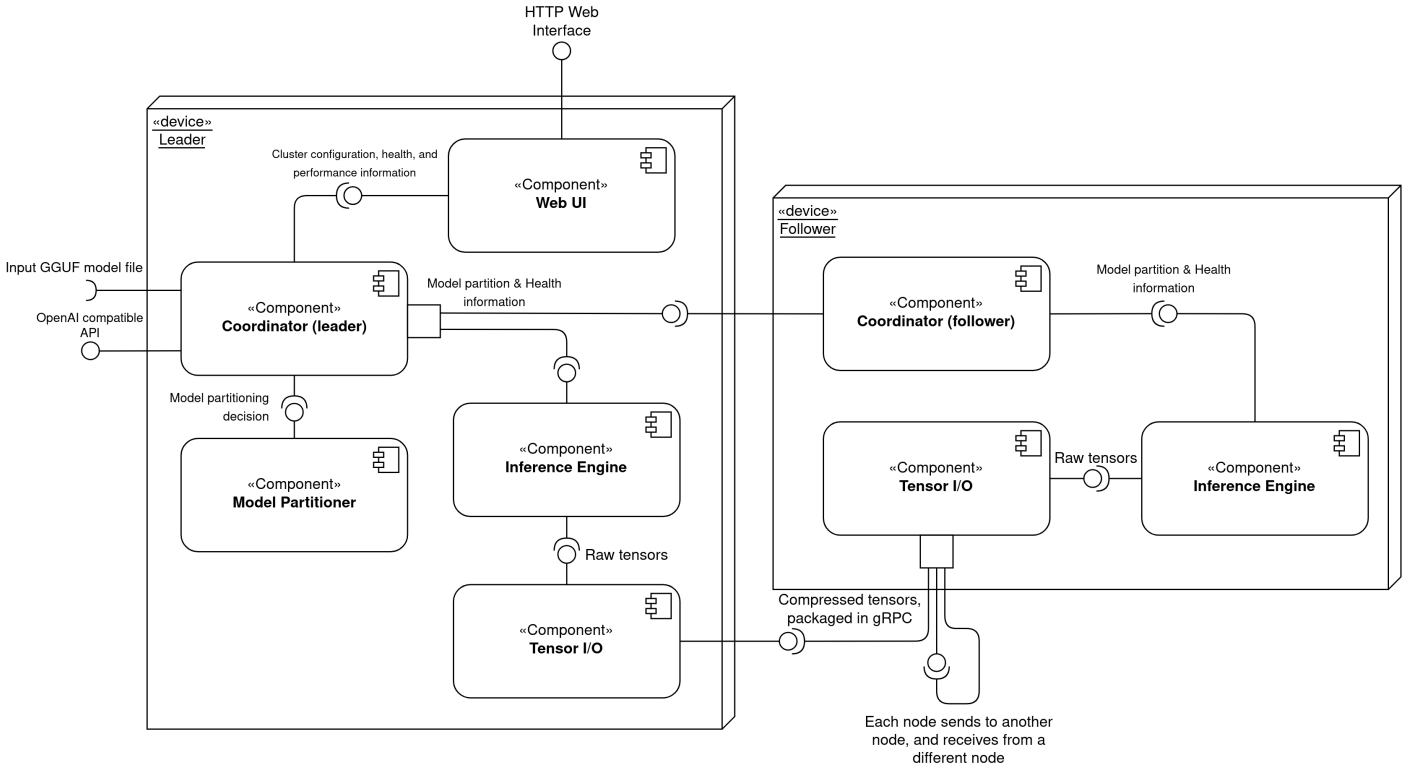


Figure 1: Component diagram.

- **Coordinator** – This component exists on all nodes, but the following responsibilities are only run on the leader node:
  - Providing an OpenAI-API compatible API to the user for use with tools like OpenCode.
  - Tracking cluster health, and detecting node failures.
  - Notifying nodes of partitioning changes.
  - Rebuilding the cluster and saving in-progress requests in the case of node failure, removal, or addition.

– Measuring node connection performance.

On a follower node, it is primarily responsible for managing the inference engine on its device, following the instructions of the leader node.

- **Model Partitioner** – This component determines how to partition the model among the participating devices. It decides at which points to cut the graph, how large each partition is, and which device gets each partition. This component is integral to ensuring that each device is assigned an appropriate partition given the device's computational and networking performance.

- **Inference Engine** – This component is the high-performance inference engine running on each device. It will handle execution of the given model partition for each device. It will call into ggml using the Rust-to-C FFI to control the execution of a device's model partition.

- **Tensor I/O** – This component handles the compressing, sending, receiving, and decompressing of tensors between devices. It will leverage gRPC for high-performance communication, with the potential to use a custom solution should gRPC fail to meet our requirements in practice.

- **Web UI** – This component handles displaying cluster information and structure to the user, as well as allowing user inputs to configure a new cluster, or modify an existing cluster.

S.2 Functionality

S.2.1 Coordinator

Functional Requirements

- **FR.S.1.1** The system shall provide the local Inference Engine with the model partition assigned to the local device.

- **FR.S.1.2** The system shall collect device health/performance information[1] from the local Inference Engine on a 5 second interval. Partially satisfies hazards H0-1 and H0-2 from the Hazard Analysis.

[1] e.g. token throughput, end-to-end partition latency, backpressure.

- **FR.S.1.3** The system shall collect the network health/performance information[2] for established connections from the Tensor I/O component on a 5 second interval. Partially satisfies hazards H0-1 and H0-2 from the Hazard Analysis.
- **FR.S.1.4** The system shall continuously scan for all other devices on the same network running Lake.
- **FR.S.1.5** The system shall provide the upstream and downstream node info to the local Tensor I/O.
- **When running on a leader node:**
  - **FR.S.1.6** The system shall send model partitions to other Coordinators on their respective devices as determined by the Model Partitioner. Partially satisfies hazard H0-2 from the Hazard Analysis.
  - **FR.S.1.7** The system shall monitor cluster performance by gathering performance data from each follower node's Coordinator. Partially satisfies hazards H0-1 and H0-2 from the Hazard Analysis.
  - **FR.S.1.8** The system shall provide the Model Partitioner with the available nodes, and gathered node performance data, to help inform optimal model cut points.
  - **FR.S.1.9** The system shall provide an HTTP API matching OpenAI's API.
  - **FR.S.1.10** The system shall detect failing follower nodes[3]. Partially satisfies hazards H2-1, H4-2 and H4-1 from the Hazard Analysis.
  - **FR.S.1.11** The system shall rebuild the cluster on a node failing or leaving. Partially satisfies hazards H2-1, H4-2 and H4-1 from the Hazard Analysis.
  - **FR.S.1.12** The system shall rebuild the cluster on a node joining.

[2] e.g. RTT, throughput, backpressure.

[3] A "failing" device is defined as severely degraded ($< 50\%$ of historical) token throughput, whether it be due to network or device resource instability.

- **FR.S.1.13** The system shall retry in-progress requests after a cluster rebuild.
- **FR.S.1.14** The system shall accept multiple concurrent requests through the HTTP API. Related to NFR.S.5.
- **FR.S.1.15** The system shall communicate the cluster graph structure to each follower.
- **FR.S.1.16** The system shall provide the Web UI with cluster information, and accept cluster configuration instructions from the Web UI.
- **FR.S.1.17** The system shall not begin inference until all selected followers have confirmed participation. Satisfies NFR.S.12.
- **FR.S.1.18** The system shall only include selected followers in a new cluster. Satisfies NFR.S.12.

- **When running on a follower node:**
  - **FR.S.1.19** The system shall accept model partition assignments from the leader node Coordinator.
  - **FR.S.1.20** The system shall, upon receiving an invite to join a cluster, request that the user confirm or decline participation. Satisfies NFR.S.12.
  - **FR.S.1.21** The system shall not partake in a cluster without user confirmation. Satisfies NFR.S.12.
  - **FR.S.1.22** The system shall not require confirmation to re-join a rebuilding cluster led by the same device, so long as the local device was already participating in the original cluster.
  - **FR.S.1.23** The system shall report collected health/performance data to the leader node Coordinator on a 5 second interval. Partially satisfies hazards H0-1 and H0-2 from the Hazard Analysis.
  - **FR.S.1.24** The system shall assume the leader node has failed if there has been no health query from the leader node for more than 60 consecutive seconds.
  - **FR.S.1.25** The system shall initiate the shutdown of the local node, if the leader node is assumed to have failed. Partially satisfies hazards H4-2 and H4-1 from the Hazard Analysis.

Rationale: The Leader node is the device the user is physically using to interact with the cluster, so failure of that device will remove the user's ability to use the cluster, whether or not the cluster survives. The architecture of Lake will allow the ability to recover from a leader failure to be implemented at a later date.

S.2.2 Model Partitioner

Functional Requirements

- **FR.S.2.1** The system shall assign all devices a partition smaller than their available GPU-accessible memory, with additional room left for runtime overhead. Related to NFR.S.4. Satisfies hazard H1-3 from the Hazard Analysis.
- **FR.S.2.2** The system shall report failure in the case that no valid partition across the cluster exists (e.g. not enough GPU-accessible memory).
- **FR.S.2.3** The system shall produce partitions that when assembled, are equivalent to the original model. Satisfies NFR.S.6.
- **FR.S.2.4** The system shall assign larger partitions to more capable devices[4]. Related to NFR.S.4.

S.2.3 Inference Engine

Functional Requirements

- **FR.S.3.1** The system shall load the model partition given by the local Coordinator into GPU-accessible memory.
- **FR.S.3.2** The system shall accept incoming tensors from Tensor I/O, and run them through the loaded partition.
- **FR.S.3.3** The system shall send output tensors to Tensor I/O.
- **FR.S.3.4** The system shall measure partition performance metrics[5]

[4]The "capabilities" of any given devices are defined via a combination of network throughput, GPU TFLOPs/TOPs, GPU-accessible memory size, and GPU-accessible memory bandwidth (GB/s).

[5]e.g. token throughput, end-to-end partition latency, backpressure.

and provide them to the local Coordinator on a 5 second interval.

- **FR.S.3.5** The system shall support processing tensors from multiple different requests concurrently. Related to NFR.S.5.

S.2.4 Tensor I/O

Functional Requirements

- **FR.S.4.1** The system shall establish connections with neighbouring nodes' Tensor I/O components.
- **FR.S.4.2** The system shall dynamically choose the compression level of outgoing tensors to maximize throughput based on available CPU-time and network throughput.
- **FR.S.4.3** The system shall accept input tensors from the upstream node, decompress them, and provide them to the Inference Engine.
- **FR.S.4.4** The system shall accept output tensors from the Inference Engine, compress them, and send them to the downstream node's Tensor I/O.
- **FR.S.4.5** The system shall measure network performance data[6], for established connections, and provide that data to the local Coordinator on a 5 second interval.
- **FR.S.4.6** The system shall maintain request association when sending tensors[7]. Related to NFR.S.5.

S.2.5 Web UI

Functional Requirements

- **FR.S.5.1** The system shall allow a user to scan for other connected devices running Lake. Partially satisfies NFR.S.2.
- **FR.S.5.2** The system shall allow a user to select connected devices to create a cluster. Partially satisfies NFR.S.2.

[6] e.g. RTT, throughput, backpressure.
[7] This is to allow tensors from different concurrent requests to be differentiated.

- **FR.S.5.3** The system shall allow a user to select an input GGUF file. Partially satisfies NFR.S.2.
- **FR.S.5.4** The system shall display an error to the user if partitioning the selected input file is infeasible given the selected devices. Partially satisfies NFR.S.1.
- **FR.S.5.5** The system shall display the performance metrics of the cluster as a whole, as well as for each individual node and connection. Partially satisfies NFR.S.1.
- **FR.S.5.6** The system shall highlight nodes or network connections causing bottlenecks (TODO: glossary bottleneck) in the cluster. Partially satisfies NFR.S.1. Satisfies hazard H4-3 from the Hazard Analysis.
- **FR.S.5.7** The system shall, upon receiving an invite to join a cluster, display a prompt for the user to either confirm or decline the invitation. Satisfies NFR.S.12.

S.2.6 Non-Functional Requirements

S.2.6.1 Usability

- **NFR.S.1 Understandability:** The user must be able to easily understand cluster information. A new user must be able to describe which devices are participating, the token throughput of the system, and which devices/connections are bottlenecks within 2 minutes of seeing the interface. Partially satisfied by FR.S.5.4, FR.S.5.5, and FR.S.5.6.
- **NFR.S.2 Controllability:** The user must be able to easily control cluster configuration. A new user must be able to create a new cluster within 2 minutes of seeing the interface. Partially satisfied by FR.S.5.1, FR.S.5.2, and FR.S.5.3.
- **NFR.S.3 Installability:** The user must be able to easily install Lake natively on Windows, macOS, and Linux. On Windows and macOS, the installation process should be automated and should require at most 10 clicks after downloading the installer. On Linux, the installation process should only require a single package manager command.

S.2.6.2 Performance

- **NFR.S.4 Model Size:** The cluster must be able to run a model with more parameters, higher precision, or larger context window than would be able to run individually on the node with the most memory. Related to FR.S.2.1, and FR.S.2.4.
- **NFR.S.5 Scalability:** The number of concurrent requests with ¡ 20% individual request performance degradation shall scale linearly with the number of participating devices. Related to FR.S.1.14, FR.S.3.5, and FR.S.4.6.
- **NFR.S.6 Intelligence:** The partitioned model must score at least 95% as well as the same model running on one machine using relevant benchmarks[8]. Satisfied by FR.S.2.3.

S.2.6.3 Look and Feel

- **NFR.S.7 Consistency:** Colors and fonts must be used consistently across the application.

S.2.6.4 Fault-Tolerance

- **NFR.S.8** The system shall reject invalid or corrupted GGUF files, and notify the user of the issue.
- **NFR.S.9** The system shall produce correct results in the face of network packet corruption and dropping up to 10% packet loss. Partially satisfies hazard H3-1 from the Hazard Analysis.

S.2.6.5 Privacy

- **NFR.S.10** The system shall not save any request data[9] after the request has completed.

[8]Relevant benchmarks are entirely dependent on what types of model are being tested. Example benchmarks: SWE-bench, FrontierMath

[9]**Request data** refers to any data sent to Lake as part of the request, or any data that would give insight into what the data sent in the request might have been.

- **NFR.S.11** The system shall not send **any** data to any device other than those included in the cluster. This includes, but is not limited to all kinds of request data, telemetery, and user data.
- **NFR.S.12** The system shall not include untrusted devices in a cluster. Satisfied by FR.S.1.17, FR.S.1.18, FR.S.1.20, FR.S.1.21, and FR.S.5.7. Satisfies hazard H5-1 from the Hazard Analysis.

S.2.6.6 Compatibility

- **NFR.S.13 Operating System:** The system shall be compatible with Windows, macOS, Linux.
- **NFR.S.14 CPU Architecture:** The system shall be compatible with x86_64 and ARM CPU architectures.
- **NFR.S.15 GPU Technology:** The system shall be compatible with NVIDIA, AMD, and Apple GPUs.

S.3 Interfaces

Lake requires the following interfaces:

- **OpenAI compatible API** – An HTTP server, following the OpenAI API structure. This will be the API used to interact with the user, via third-party tools of the user's choice that accept the OpenAI API (e.g. OpenCode).
- **GGUF input model** – An input file using the GGUF file format. This contains the model Lake will partition and run.
- **Rust-to-C FFI** – The Rust-to-C foreign function interface will be used to call ggml from the Inference Engine.
- **gRPC between nodes** – A gRPC-based API used to communicate between nodes.
- **REST API between Coordinator and Web UI** – A REST API for communication between the leader Coordinator and the Web UI.

S.4 Normal Operation

1. User opens Lake instances on all participating machines.
2. User creates a Lake cluster using the UI on one of the devices, and provides a GGUF file. This device becomes the leader node.
3. Leader node checks connectivity with follower nodes, and collects performance metrics for each node.
4. Partitioner devises a partitioning scheme based on the collected metrics.
5. Leader node sends the model partitions to their respective devices.
6. Neighbouring nodes on the partition graph establish direct P2P connections.
7. User initiates inference request by providing a prompt via the OpenAI compatible API.
8. Leader node sends the prompt tokens to the inference engine corresponding to the first node of the graph, initiating prefill.
9. Prompt tokens propagate through the graph until prefill completes, and the first output token is generated.
10. Output tokens are repeatedly fed back into the top of the graph, generating a new token on each iteration until the response is complete.
11. As the response generates, the user receives the generated output via the OpenAI compatible API.
12. User initiates a shutdown of the Lake cluster.
13. Each node unloads its portion of the graph.
14. Leader node saves the cluster configuration to disk.
15. All nodes sever their connections, and all Lake instances terminate.

S.5 Undesired Event Handling

- **Model is too large for the available memory in the cluster** – The Partitioner reports a partitioning failure, and this error is communicated to the user via the Web UI.
- **Follower node fails during inference** – Leader node re-tries inference, if it continues to fail, the leader rebuilds the cluster, excluding the problematic device. Associated with hazard H2-1 in the Hazard Analysis.
- **Tensors are corrupted on transfer between nodes** – The receiving Tensor I/O requests that the tensor is retransmitted from the source Tensor I/O. Associated with hazard H3-1 in the Hazard Analysis.
- **Connection between two nodes is lost** – The nodes in question attempt to re-establish a connection. If this fails, the leader must rebuild the cluster, excluding the failed link. Associated with hazard H4-1 in the Hazard Analysis.
- **Input GGUF file is corrupted or fails to load** – Refuse to begin inference, and inform the user of the corrupted or faulty file.
- **A request is made with a larger prompt than the cluster has memory for** – If there are existing requests running, wait for them to complete, then try again. If there is still not enough memory, return an error for the request, and instruct the user to shorten their prompt or context window.
- **A rebuilt cluster is too small to hold the model after a node leaves** – The Partitioner reports a partitioning failure, and this error is communicated to the user via the Web UI.
- **A user attempts to create a cluster with a node without a GPU** – The Web UI displays an error message outlining the requirement for all nodes to have a GPU.

S.6 Detailed Usage Scenarios

[Full use cases for the scenarios introduced in G.5, one per scenario, in the structure below. The main success scenario should be numbered so extensions

can reference its steps. —TPLT]
Use Case Name: Name
Primary Actor: Actor
Stakeholders and Interests:

- Stakeholder: interest.

Preconditions:

- Precondition.

Postconditions (Success Guarantees):

- Postcondition.

Postconditions (Failure Guarantees):

- Postcondition.

Main Success Scenario (Basic Flow):

1. Step.

Extensions:

- **Na.** Alternative or failure at step N, and how it is handled.

Justification: Why this scenario matters.

S.7 Prioritization

[Every requirement from S.2 and the Environmental Requirements table, with a MoSCoW priority and the reason for it. Group the tables by component. The Date column records when the priority was last decided. —TPLT]
Priorities follow MoSCoW: **M** must have, **S** should have, **C** could have, **W** won't have (this release).

Component Name

ID	Requirement Name	Priority	Reasoning	Date
FR.S.1.1	Name	M	Reasoning	YYYY-MM-DD
FR.S.1.2	Name	S	Reasoning	YYYY-MM-DD

Table 4: Prioritization of Component Name requirements

S.8 Verification and Acceptance Criteria

[How each class of requirement will be shown to be satisfied: unit and integration tests, measurements, user studies, inspections. Make the criteria concrete enough that the V&V Plan can be derived from them. Every requirement in S.2 should map to at least one criterion here. —TPLT]

Validation Overview:

Verification Approach by Requirement Class:

- **Class of requirements** – How it will be verified and what counts as acceptance.

S.9 Formalizations

Two parts of Lake are formalized. S.9.2 defines what counts as a valid partitioning scheme for the model across the cluster, which every other requirement depends on. S.9.3 specifies, as a finite state machine, how the system responds when nodes join and leave the cluster. This is the precise form of the Dynamic Allocation goal.

S.9.1 Notation

For integers a and b, $a..b = \{\, n \in \mathbb{N} \mid a \leqslant n \leqslant b \,\}$, which is empty when $a > b$.

Table 5: Symbols used in the formalization

Symbol	Type (unit)	Meaning
N	$\mathbb{N}$, $N \geqslant 1$	Number of partition units in the model (embedding, transformer layers and output head), in execution order.
L	$L = 1..N$	The set of partition units.
D	finite set	The nodes currently in the cluster: devices the user has selected (FR.S.5.2) and that have confirmed participation (FR.S.1.21).
m_ℓ	$\mathbb{N}$ (bytes)	Memory needed to hold unit ℓ's weights and its KV cache for $CONTEXT_MAX$ tokens.
M_d	$\mathbb{N}$ (bytes)	GPU-accessible memory free on node d when it joins the cluster, minus the runtime overhead reserved by FR.S.2.1.

The formalization assumes that M_d does not change while d is in the cluster, and that $join(d)$ only occurs for $d \notin D$ and $leave(d)$ only for $d \in D$.

S.9.2 Valid Partitioning Schemes

Definition S.9.2.1 (Partitioning scheme)**.** A *partitioning scheme* is a finite sequence of partitions

$$P = \langle (d_1, a_1, b_1), \ldots, (d_k, a_k, b_k) \rangle, \qquad k \geqslant 1,$$

where d_i is a node and $a_i, b_i \in \mathbb{N}$. Partition i consists of units $a_i..b_i$, runs on node d_i, and passes its output to partition $i + 1$. The nodes of P are $dev(P) = \{d_1, \ldots, d_k\}$.

Definition S.9.2.2 (Valid partitioning scheme)**.** P is *valid* over cluster D, written $P \in \mathcal{V}(D)$, if and only if:

(V1) $dev(P) \subseteq D$ and $d_i \neq d_j$ for all $1 \leqslant i < j \leqslant k$;

(V2) $a_1 = 1$ and $b_k = N$;

(V3) $a_i \leqslant b_i$ for all $i \in 1..k$;

(V4) $a_{i+1} = b_i + 1$ for all $i \in 1..k - 1$;

(V5) $\sum_{\ell=a_i}^{b_i} m_\ell \leqslant M_{d_i}$ for all $i \in 1..k$.

(V1) gives each node at most one partition. (V2)–(V4) make the partitions consecutive, non-empty blocks of units in execution order. (V5) says each node has room for its partition. The Model Partitioner must return a member of $\mathcal{V}(D)$ whenever $\mathcal{V}(D) \neq \emptyset$, and must report failure otherwise (FR.S.2.2).

Lemma S.9.2.3 (Exact cover)**.** *If $P \in \mathcal{V}(D)$, every unit $\ell \in L$ belongs to exactly one partition. That is, the sets $a_i..b_i$ for $i \in 1..k$ are non-empty, pairwise disjoint, and their union is L.*

Proof. Let $b_0 = 0$. By (V2) and (V4), $a_i = b_{i-1} + 1$ for all $i \in 1..k$, so partition i consists of units $(b_{i-1}+1)..b_i$. By (V3) these sets are non-empty, so $0 = b_0 < b_1 < \cdots < b_k = N$. Each $\ell \in L$ therefore satisfies $b_{i-1} < \ell \leqslant b_i$ for exactly one i, so it belongs to exactly one partition. □

S.9.3 Allocation State Machine

The system's response to nodes joining and leaving the cluster is a finite state machine (Q, Σ, δ, q_0) whose transitions are guarded by predicates over the cluster D and the active partitioning scheme P.

S.9.3.1 States.

$Q = \{\mathsf{U}, \mathsf{A}, \mathsf{I}, \mathsf{G}, \mathsf{R}, \mathsf{X}\}$ with $q_0 = \mathsf{U}$:

- U (Unloaded): no model is loaded.
- A (Allocating): a target scheme P' is being loaded, and no request is in progress.
- I (Idle): the model is loaded on scheme P.
- G (Generating): a request is being served on P.
- R (Recovering): a node in $dev(P)$ left during a request. The request is stopped while P' is loaded, then retried (FR.S.1.13).
- X (Insufficient): $\mathcal{V}(D) = \emptyset$, so the cluster cannot hold the model.

S.9.3.2 Events. $\Sigma = \{load, unload, req, fin, done\} \cup \{join(d), leave(d)\}$:

- *load* and *unload*: the user creates a cluster with a model, or shuts it down.
- *req* and fin: a request is submitted, or finishes.
- *done*: the system finishes loading P'.
- $join(d)$: a device the user selected confirms participation and is added to the cluster.
- $leave(d)$: follower d leaves, or is detected as failing (FR.S.1.10).

This finite state machine only models the behaviour of follower nodes. If the leader node is detected as failed (FR.S.1.24), the followers shut down and the session ends (FR.S.1.25), which has the same effect as *unload*.

S.9.3.3 Guards. Let D' be the cluster after the event: $D \cup \{d\}$ for $join(d)$, $D \setminus \{d\}$ for $leave(d)$, and D otherwise. Then

$$\varphi \equiv \mathcal{V}(D') \neq \emptyset \qquad \text{and} \qquad \psi \equiv d \in dev(P).$$

Whenever the machine enters A or R, it selects a new target $P' \in \mathcal{V}(D')$.

S.9.3.4 Transitions. Table 6 defines δ. Combinations it does not list are handled as follows:

- *unload* moves every state to U and aborts any request in progress.
- $leave(d)$ with $\neg\psi$ in I or G removes d from D and leaves the state unchanged.
- *req* is rejected with an error in U and X. *req* in A, G or R, and $join(d)$ in G or R, are queued and fire in arrival order once I is entered. The machine therefore serves one request at a time, and concurrent requests (FR.S.1.14) are not modelled.
- Any other combination leaves the state unchanged.

Table 6: Transition function δ of the allocation state machine

#	State	Event [guard]	Next	Effect
1	U	*load* $[\varphi]$	A	Start loading P'.
2	U	*load* $[\neg\varphi]$	X	Report a partitioning failure.
3	A	*done*	I	$P := P'$.
4	A	$join(d)$, $leave(d)$ $[\varphi]$	A	Reselect P' and restart loading.
5	A	$join(d)$, $leave(d)$ $[\neg\varphi]$	X	Report a partitioning failure.
6	I	*req*	G	Start generating.
7	I	$join(d)$	A	Rebuild the cluster over D'.
8	I	$leave(d)$ $[\psi \wedge \varphi]$	A	Rebuild the cluster over D'.
9	I	$leave(d)$ $[\psi \wedge \neg\varphi]$	X	Report a partitioning failure.
10	G	*fin*	I	Return the response.
11	G	$leave(d)$ $[\psi \wedge \varphi]$	R	Stop the request.
12	G	$leave(d)$ $[\psi \wedge \neg\varphi]$	X	Abort the request with an error.
13	R	*done*	G	$P := P'$, retry the request.
14	R	$leave(d)$ $[\varphi]$	R	Reselect P' and restart loading.
15	R	$leave(d)$ $[\neg\varphi]$	X	Abort the request with an error.
16	X	$join(d)$ $[\varphi]$	A	Start loading P'.

Row 7 needs no guard. Conditions (V1)–(V5) refer to D only through $dev(P) \subseteq D$, so a scheme valid over D stays valid over $D \cup \{d\}$. For the same reason, removing a node outside $dev(P)$ keeps P valid.

S.9.3.5 Invariant. In states I and G, $P \in \mathcal{V}(D)$. P is only assigned in rows 3 and 13, from a P' that is reselected after every change to the cluster. The only change that keeps the machine in I or G removes a node outside $dev(P)$.

S.10 Likely Changes to Requirements

- **LC.1** A Lake cluster cannot recover from a failing leader node (see FR.S.1.25). In the future, the ability to recover from a failing leader node may be added.
- **LC.2** Lake only supports desktop operating systems (Windows, macOS, Linux) (see NFR.S.13). Support for mobile devices (iOS, Android)

may be added once other goals and requirements have been met.

- **LC.3** Lake assumes the local area network being used to connect the devices is private and trusted (See Hazard Analysis Section 4 - Critical Assumptions). Lake's may be extended with additional security hardening to allow clusters to run on public or untrusted networks.
- **LC.4** Lake requires that each device manually confirm joining a cluster (see FR.S.1.17 and FR.S.1.21). In the future, a trust mechanism may be introduced, which would allow devices to automatically join clusters created by trusted devices without requiring confirmation.
- **LC.5** Requirement thresholds and intervals will likely change, as real-world usage gives better estimates for reasonable threshold values.
- **LC.6** It is likely that future versions of Lake will support a headless mode where all UI interactions are performed through a command line interface. This would be useful for users wishing to use headless servers as nodes.

S.11 Unlikely Changes to Requirements

- **UC.1** Model training and fine-tuning support is unlikely to be added to Lake. Lake is primarily an inference framework, so the addition of training would be out of scope for such a project.
- **UC.2** Additional input model formats aside from GGUF (TODO: glossary) are unlikely to be added, as GGUF is one of the most common standards for LLMs (TODO: glossary). Additionally, models in other formats can be converted to GGUF so there is little reason for Lake to support them directly.
- **UC.3** A primary goal of Lake is privacy (see NFR.S.11), so modifications that would compromise privacy like incorporating cloud-based services are unlikely.
- **UC.4** Even though one of Lake's goals is to run larger models, integrating quantization tools into Lake is unlikely. Lake's scope is focused on distributed inference, and quantization tools would better fit in the scope of an independent project.

(P) Project

[The Project chapter covers how the system will be built: people, imposed choices, schedule, deliverables, risks, and how requirements will be managed over the project's life. —TPLT]

P.1 Roles and Personnel

[Each team member and their role, plus the supervisor and TA. Keep this consistent with the Development Plan. —TPLT]

- **Role (Name)** – Responsibilities.

P.2 Imposed Technical Choices

[Technologies that are fixed before design begins and why: languages, frameworks, hardware, platforms mandated by the client, supervisor, or environment. Choices the team is still free to make do not belong here. —TPLT]

- **Choice** – What is imposed and by whom.

P.3 Schedule and Milestones

[The major milestones and their dates. Course deliverables are a starting point; add project-specific milestones such as prototypes or integration points. —TPLT]

Milestone	Due Date
Milestone	Date

Table 7: Project Milestones and Due Dates

P.4 Tasks and Deliverables

[The work breakdown: each task, what it produces, and which milestone it feeds. Tasks should be small enough to assign to one or two people. —TPLT]

Task	Description / Steps	Deliverable	Due Date
Task	Description	Deliverable	Date

Table 8: Tasks and Deliverables

P.5 Required Technology Elements

[Everything the project needs to obtain or build before the system can be delivered: libraries, hardware, datasets, services, licences, compute. Note anything not yet secured. —TPLT]

- **Element** – Purpose and status.

P.6 Risk and Mitigation Analysis

[Project risks (schedule, technical, people, external dependencies), each with its likelihood, impact, and mitigation. System hazards belong in the Hazard Analysis, not here. —TPLT]

- **Risk: Name**
  - Description:
  - Mitigation:

P.7 Requirements Process and Report

[How requirements were elicited and how they will be maintained: who was consulted, through which channels, how changes are tracked, and how this document will be kept in sync with the Hazard Analysis, V&V Plan and design. —TPLT]

- **Elicitation** –
- **Communication** –
- **Change Tracking** –
- **Process Updates** –

Safety and Security Requirements

[Port the requirements from the Safety and Security Requirements section of the Hazard Analysis so this document is the single list of requirements. Keep the labels identical in both documents. —TPLT]

Functional Safety Requirements

- **FR.Sa.1** The system shall ...

Non-Functional Safety Requirements

- **NFR.Sa.1** The system shall ...

Requirement Dependencies

[A traceability diagram or matrix showing which requirements depend on which. Use the landscape environment below for a wide figure. —TPLT]

[Cite sources with \citep{key} or \citet{key}; entries live in refs/References.bib. The References section appears once at least one source is cited. —TPLT]

1 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

1.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

1.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

1.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

2 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?